DATABASE MANAGEMENT SYSTEMS

CT 652 | BE / BCT | Year III / Part II

Tribhuvan University — Institute of Engineering

CHAPTER 8: CRASH RECOVERY

Question Bank with Comprehensive Answers — Failure Types, Storage Structure, Log-Based Recovery, Deferred & Immediate Modification, Checkpoints, Shadow Paging

SYLLABUS OVERVIEW — Unit 8

Unit	Topic	Hours	Marks
1	Introduction	3 hrs	4
2	Data Models	7 hrs	12
3	Relational Languages & Model	7 hrs	12
4	Constraints & Normalization	6 hrs	12
5	Query Processing & Optimization	4 hrs	8
6	File Structure & Hashing	4 hrs	8
7	Transactions & Concurrency Control	6 hrs	12
8	Crash Recovery	4 hrs	6
9	Advanced Database Concepts	4 hrs	6
Total		**45 hrs**	**80**

Chapter 8 Topics (4 Hours | 6 Marks)

- 8.1 Failure Classification (transaction, system, disk)
- 8.2 Storage Structure (volatile, non-volatile, stable)
- 8.3 Recovery and Atomicity
- 8.4 Log-Based Recovery — log records, WAL
- 8.5 Deferred Database Modification
- 8.6 Immediate Database Modification — undo and redo
- 8.7 Checkpoints
- 8.8 Shadow Paging
- 8.9 Recovery with Concurrent Transactions

CHAPTER 8 — Questions & Comprehensive Answers

Q: What are different types of failures in a database system? Explain each with examples.
Asked: ALL years 2070–2082 — asked in every single exam | Marks: [4]

Answer

Crash Recovery

Crash recovery is the process of restoring a database to a consistent state after a failure. Failures can occur at any point during transaction execution, potentially leaving the database in an inconsistent state. The recovery system must ensure both atomicity (incomplete transactions are undone) and durability (committed transactions survive).

Types of Failures

1. Transaction Failure

A transaction failure occurs when a single transaction cannot complete its normal execution. There are two sub-types:

- Logical error: The transaction cannot complete because of some internal condition — overflow, invalid data, data not found, integrity constraint violation, or explicit user abort. Example: A transfer transaction fails because the account does not have sufficient balance.
- System error: The DBMS terminates an active transaction for system reasons, such as a deadlock. The transaction is aborted and must be restarted. Example: Transaction T1 is aborted to break a deadlock with T2.

```
Recovery action: ROLLBACK — undo all changes made by the failed transaction
  → Restore all modified data items to their values before the transaction
began
  → Uses the UNDO log records
```

2. System Crash (Software Failure)

A system crash occurs when the computer system itself fails — power outage, operating system crash, hardware malfunction, or software bug. The contents of volatile (main memory) storage are lost, but non-volatile storage (disk) is assumed to survive intact.

- All transactions that were active (not yet committed) at the time of the crash must be rolled back
- All transactions that had committed before the crash must have their changes reflected in stable storage
- The database may be in an inconsistent state because some buffer pages (in-memory copies of disk blocks) had not yet been written to disk

```
Example: Transaction T1 updates A and B.
  T1 writes A to buffer, then writes A to disk
  System crashes BEFORE T1 writes B to disk
  → Disk: A is updated, B is NOT updated → INCONSISTENT
```

```
Recovery action: REDO if T1 committed before crash; UNDO if T1 was active at
crash
```

3. Disk Failure (Media Failure)

A disk failure (head crash or block corruption) results in the destruction of some or all of the disk storage. This is the most severe type of failure because even non-volatile storage is damaged. Recovery requires restoring from a backup copy (dump) and replaying the log.

- Head crash: read/write head physically damages disk platters — data on affected tracks is permanently lost
- Controller failure: the disk controller malfunctions, causing data corruption
- Bad sectors: portions of the disk become unreadable due to magnetic deterioration

```
Recovery action:
  1. Restore the most recent database backup (dump) from stable storage
(tape/backup disk)
  2. Replay (REDO) all committed transactions from the log since the backup
  → Requires archival log files and periodic database dumps
```

Failure Type	Cause	Memory Lost?	Disk Lost?	Recovery Method
Transaction Failure	Logical error, deadlock, abort	Partially	No	Rollback (UNDO) the transaction
System Crash	Power outage, OS crash, hardware fault	Yes (volatile)	No	Redo committed; Undo active txns at crash
Disk Failure	Head crash, bad sectors, controller fault	Yes	Yes (partial/full)	Restore backup + replay archived log

---

Q: Explain the storage structure used in database systems: volatile storage, non-volatile storage, and stable storage.

Asked: 2079 Ashwin; 2079 Jestha; 2078 Poush; 2076; 2075 Bhadra; 2075 Baisakh; 2073 Magh; 2071 Magh; 2070 Magh | Marks: [4]

Answer

Storage Structure

Database systems use different types of storage media with different speed, capacity, and reliability characteristics. Understanding the storage hierarchy is crucial for crash recovery design.

1. Volatile Storage

Volatile storage loses its contents when power is removed (system failure or power outage). It is the fastest type of storage and is used for in-memory processing.

- Examples: Main memory (RAM), CPU caches, CPU registers
- Speed: Extremely fast (nanoseconds access time)
- Capacity: Limited (typically GBs of RAM)

- Durability: Does NOT survive system crashes
- Use in DBMS: Buffer pool (in-memory cache of disk pages), currently executing transactions, lock tables

2. Non-Volatile Storage

Non-volatile storage retains its contents when power is removed — data survives system crashes. However, it can fail due to hardware faults (disk head crashes, bad sectors).

- Examples: Magnetic disk (HDD), SSD, magnetic tape, optical disks
- Speed: Slower than volatile storage (milliseconds for HDD)
- Capacity: Large (TBs of disk)
- Durability: Survives system crashes; NOT immune to disk failure
- Use in DBMS: Primary database files, index files, log files

3. Stable Storage

Stable storage is a theoretical concept — information stored in stable storage is NEVER lost. In practice, it is approximated by writing data to multiple independent, geographically distributed non-volatile storage devices. Even if one copy is destroyed, the others survive.

- Implementation: RAID (mirroring), multiple copies on different disks at different locations, backup tapes in a different building
- Write protocol: Data is written to all copies. A write is only considered complete when all copies confirm it.
- Use in DBMS: Transaction log (write-ahead log) — the log must be in stable storage so that recovery is always possible

```
Stability assumption in recovery theory:
  Stable storage survives ALL failures considered in the recovery model.
  The log is always in stable storage — even after a crash, the log is intact.
  This is what makes log-based recovery work.
```

Storage Type	Examples	Survives Crash?	Survives Disk Failure?	Speed
Volatile	RAM, CPU cache	No — data lost on crash	No	Fastest (ns)
Non-volatile	HDD, SSD, tape	Yes	No — disk can fail	Medium (ms)
Stable	Mirrored disks, RAID, off-site backup	Yes	Yes (multiple copies)	Slowest (write to all copies)

Q: What is log-based recovery? Explain the log record format, write-ahead logging (WAL) rule, and the importance of the log.

Asked: ALL years 2070–2082 | Marks: [4]

Answer

<u>Log-Based Recovery</u>

Log-based recovery is the standard technique used by almost all production database systems. It maintains a log (also called the journal or audit trail) — a sequential record of all database modifications made by transactions, written to stable storage. This log is used to recover the database after a failure.

Log Record Format

The log is a sequence of log records. Each log record documents one step of a transaction. The standard log record types are:

```
<Ti, start>                    — Ti has started
<Ti, Xj, V1, V2>              — Ti wrote data item Xj;
                                  V1 = old value (before-image)
                                  V2 = new value (after-image)
<Ti, commit>                   — Ti has committed
<Ti, abort>                    — Ti has aborted
<checkpoint L>                 — checkpoint record;
                                  L = list of active transactions at
checkpoint time
```

Write-Ahead Logging (WAL) Rule

The Write-Ahead Logging rule is the fundamental rule of log-based recovery. It has two parts:

- WAL Rule 1 (for undo): Before a data item X is written to the database (on disk), the log record <Ti, X, old_value, new_value> MUST be written to stable storage (log on disk)
- WAL Rule 2 (for redo): Before a transaction Ti can commit, ALL its log records (including <Ti, commit>) must be written to stable storage

```
WHY WAL?
  Rule 1 ensures: if a crash occurs after the data is written but before the
log is written,
                  we have no way to undo the change → WAL prevents this
  Rule 2 ensures: if a crash occurs after Ti's data is written but before
<Ti,commit>,
                  we can redo Ti's changes on recovery

In other words: The log must always be 'ahead of' the actual database changes.
```

Recovery Operations

```
UNDO(Ti):  Restore all data items modified by Ti to their old (before-image)
values
           → Used when Ti was active (not committed) at the time of crash
           → Scans log BACKWARDS from end to find Ti's writes
           → For each <Ti, Xj, V1, V2>: set Xj = V1 (old value)
           → Writes <Ti, abort> to log when done

REDO(Ti):  Re-apply all data writes of Ti using their new (after-image) values
           → Used when Ti committed but changes may not be on disk
           → Scans log FORWARDS from the start to find Ti's writes
           → For each <Ti, Xj, V1, V2>: set Xj = V2 (new value)
           → Idempotent: applying redo multiple times gives same result
```

Q: Explain deferred database modification with a log example. When are changes actually written to disk? What happens during recovery?

Asked: 2081 Chaitra; 2079 Chaitra; 2079 Jestha; 2078 Chaitra; 2078 Baishakh; 2076; 2074; 2072 Ashwin; 2071 Bhadra; 2070 | Marks: [4]

Answer

Deferred Database Modification

In the deferred modification scheme, a transaction does NOT write any changes to the actual database (on disk) until it commits. All modifications are recorded only in the log and kept in memory buffers. The actual database update is deferred (postponed) until commit time. Only the new values (after-images) need to be stored in log records — no old values are needed for this scheme.

Log Record Format (Deferred)

```
Deferred modification log records (no old values needed):
  <Ti, start>
  <Ti, Xj, V2>     ← only new value V2; old value not recorded
  <Ti, commit>     ← ONLY NOW are changes written to database
```

Recovery Rules for Deferred Modification

```
On crash, for each transaction Ti:

  If log has <Ti, commit>:
    → Ti committed. Its changes MAY or MAY NOT be on disk.
    → REDO Ti: apply all its log writes (forward scan)
    → Re-execute <Ti, Xj, V2> records in log order

  If log has <Ti, start> but NO <Ti, commit>:
    → Ti was active at crash time. It never committed.
    → Because deferred: NO changes were written to disk anyway
    → NO action needed (nothing to undo — disk is clean)
    → Simply ignore Ti

Summary: Deferred modification only needs REDO — never UNDO
```

Worked Example — Deferred Modification

Transactions T1 and T2 execute concurrently. T1 reads A, writes A, reads B, writes B. T2 reads A, writes A. Initial values: A=1000, B=2000.

```
LOG (written to stable storage):          DATABASE (disk):
__________________________________        __________________

<T1, start>                               A=1000  B=2000
<T2, start>
<T1, A, 950>           ← T1 writes A (only in log; disk unchanged)
<T1, B, 2050>          ← T1 writes B (only in log; disk unchanged)
<T1, commit>           ← NOW T1's changes are written to disk
                                          A=950   B=2050
<T2, A, 900>           ← T2 writes A (only in log)
```

```
                                        CRASH!

RECOVERY after crash:
  Scan log forward:
    T1: <T1,start> + <T1,commit> → REDO T1: set A=950, B=2050 ✓
    T2: <T2,start> but NO <T2,commit> → T2 not committed
        Deferred: nothing to undo (T2 wrote nothing to disk)
        Ignore T2

  Final state after recovery: A=950, B=2050 ✓
```

Different Crash Scenarios

Crash Point	T1 Status	T2 Status	Recovery Action
Before <T1,commit>	Not committed	Not committed	Nothing to do — disk untouched
After <T1,commit>, before T2 crash	Committed	Not committed	Redo T1 (if not on disk); Ignore T2
After both commit	Both committed	Both committed	Redo both if needed

Key advantage of deferred modification: Simple recovery — only REDO needed, never UNDO.
Key limitation: Transaction must buffer all its changes in memory until commit. For long transactions or large data, this requires a lot of memory.
Old values (V1) not needed in log — saves log space.

Q: Explain immediate database modification. What are undo and redo operations? Trace an example log with multiple crash scenarios.
Asked: 2082; 2081 Chaitra; 2080; 2079 Ashwin; 2078 Chaitra; 2077; 2075; 2073 Magh; 2072 Magh; 2070 Magh — all years | Marks: [4]

Answer

Immediate Database Modification

In the immediate modification scheme, a transaction CAN write its changes to the actual database (disk) at any time, even BEFORE it commits. This allows the buffer manager to write dirty pages to disk whenever convenient (using the steal/no-force buffer policy). The log must record BOTH old and new values.

Log Record Format (Immediate)

```
Immediate modification log records (both old AND new values):
  <Ti, start>
  <Ti, Xj, V1, V2>   ← V1=old value, V2=new value (BOTH recorded)
  <Ti, commit>

WAL rule strictly enforced: log record written to stable storage
```

```
BEFORE the actual database write goes to disk.
```

Recovery Rules for Immediate Modification

```
On crash, for each transaction Ti in the log:

  If log has <Ti, commit>:
    → Ti committed. Changes MAY or MAY NOT be on disk (buffer eviction).
    → REDO Ti: scan log FORWARD, re-apply all <Ti, Xj, V1, V2> with V2

  If log has <Ti, start> but NO <Ti, commit>:
    → Ti was active at crash. Changes MAY be partially on disk.
    → UNDO Ti: scan log BACKWARD, apply all <Ti, Xj, V1, V2> with V1
    → Restore all of Ti's items to their old values

Recovery procedure:
  Phase 1 (REDO phase): Scan log forward from start; redo all committed Tis
  Phase 2 (UNDO phase): Scan log backward; undo all active (uncommitted) Tis
```

Worked Example — Immediate Modification

Initial: A=1000, B=2000. T1 transfers 50 from A to B. T2 transfers 100 from A elsewhere.

```
LOG (stable storage):                    DATABASE (disk, written anytime):
_________________________________        _________________________________

<T1, start>                              A=1000  B=2000
<T2, start>
<T1, A, 1000, 950>   ← T1 writes A
                                         A=950   (buffer written to disk)
<T2, A, 950,  850>   ← T2 writes A
                                         A=850   (buffer written to disk)
<T1, B, 2000, 2050>  ← T1 writes B
<T1, commit>                             B=2050  (T1 committed, B on disk)

  *** CRASH HERE ***

RECOVERY:
  Redo set: {T1}  (has <T1,commit>)
  Undo set: {T2}  (has <T2,start> but no <T2,commit>)

  REDO phase (forward scan):
    <T1,A,1000,950>: set A=950   (idempotent — might already be 950)
    <T1,B,2000,2050>: set B=2050

  UNDO phase (backward scan):
    <T2,A,950,850>: set A=950   (restore T2's write using old value V1)
    Write <T2,abort> to log

FINAL STATE after recovery: A=950, B=2050 ✓
  (T1's transfer of 50 from A to B is preserved)
  (T2's uncommitted changes are rolled back)
```

Scenarios at Different Crash Points

Crash Point	T1	T2	Action for T1	Action for T2

Before <T1,A,1000,950>	Active	Active	Undo T1 (nothing written)	Undo T2 (nothing written)
After T2 writes A=850, before T1 commits	Active	Active	Undo T1: A → 1000	Undo T2: A → 1000 (apply both undos in reverse log order)
After <T1,commit>, before T2 crash	Committed	Active	Redo T1: A=950,B=2050	Undo T2: A → 950
After both commit	Both committed	Committed	Redo both	Redo both

Aspect	Deferred Modification	Immediate Modification
When written to disk	Only at commit (deferred)	Any time (before or after commit)
Log records	<Ti, Xj, V2> — new value only	<Ti, Xj, V1, V2> — old AND new value
Recovery operations	REDO only (never UNDO)	Both REDO (committed) and UNDO (active)
Memory requirement	Buffer all changes until commit	Buffer pages can be evicted anytime
Complexity	Simpler	More complex but more flexible
Used in practice	Rare	Standard (most production DBMS)

Q: Explain checkpoint-based recovery. Why are checkpoints needed? Describe the checkpoint procedure and how recovery uses checkpoints.
Asked: 2081 Ashwin; 2080 Ashwin; 2079 Ashwin; 2078 Poush; 2077; 2075 Baisakh; 2074; 2071 Magh | Marks: [4]

Answer

Problem Without Checkpoints

Without checkpoints, on every crash recovery the system must scan the ENTIRE log from the beginning — which could be days or weeks of history. Even transactions that committed long ago might need to be redone (their changes might not have been written to disk at that ancient time). This makes recovery extremely slow.

Checkpoint

A checkpoint is a synchronization point between the database and the log. At a checkpoint, the DBMS ensures that all changes in the buffer pool (in-memory) are written to disk (flushed), and a special <checkpoint> log record is written to the log. After a crash, recovery only needs to scan the log from the most recent checkpoint — not from the beginning.

Checkpoint Procedure

```
To take a checkpoint:
  1. Write to log all log records currently in the volatile log buffer
```

```
      (flush log buffer to stable storage)
   2. Write all modified buffer pages (dirty pages) to disk
      (flush buffer pool to disk — this is the expensive step)
   3. Write a <checkpoint L> record to the log, where
      L = list of all transactions active at checkpoint time
   4. Flush the log again (write <checkpoint> record to stable storage)

After checkpoint: All committed transactions' changes are guaranteed on disk.
  → On recovery, need not redo any transaction that committed before checkpoint.
```

Recovery Using Checkpoints

```
                                Log timeline:

            ──────────────────────────────────────────────────────────→ time
                        |                   |                       |
                  (old log)        [CHECKPOINT]              [CRASH]
                                        |                         |
                                  L={T2,T3}           active: T3,T4,T5

         T1: |──────commit──────|                (committed before checkpoint)
      T2: |──────────────────────commit──| (started before, commit after chkpt)
      T3: |──────────────────────────────────────────|   (started before, active at
                                        crash)
        T4:                     |────commit──|       (started after checkpoint,
                                        committed)
      T5:                     |──────────────────────|   (started after checkpoint,
                                        active)
```

```
Recovery decision for each transaction:

  T1: Committed BEFORE checkpoint.
      Its changes are guaranteed on disk (checkpointed).
      → No action needed (neither redo nor undo)

  T2: Started before checkpoint; committed AFTER checkpoint.
      Its changes MAY or MAY NOT be on disk at crash.
      → REDO T2 (scan from checkpoint forward)

  T3: Started before checkpoint; ACTIVE at crash (no commit).
      Its changes may be partially on disk.
      → UNDO T3 (scan backward from crash point)

  T4: Started after checkpoint; committed before crash.
      Changes may not be on disk.
      → REDO T4

  T5: Started after checkpoint; ACTIVE at crash (no commit).
      → UNDO T5

Redo set (scan log forward from checkpoint): {T2, T4}
Undo set (scan log backward from crash): {T3, T5}
```

General Recovery Algorithm with Checkpoints

```
1. Find the most recent <checkpoint L> in the log
   L = list of active transactions at checkpoint

2. Initialize:
   redo-list = {} (transactions to redo)
   undo-list = L  (start with transactions active at checkpoint)

3. Scan log FORWARD from checkpoint to crash point:
   For each <Ti, start>: add Ti to undo-list
   For each <Ti, commit>: move Ti from undo-list to redo-list
   For each <Ti, abort>: remove Ti from undo-list

4. REDO phase: scan log forward from checkpoint
   For each <Ti, Xj, V1, V2> where Ti in redo-list:
     Set Xj = V2 (apply new value)

5. UNDO phase: scan log backward from crash point
   For each <Ti, Xj, V1, V2> where Ti in undo-list:
     Set Xj = V1 (restore old value)
   For each Ti in undo-list: write <Ti, abort> to log

This is the standard recovery algorithm used in most DBMS.
```

Fuzzy Checkpoint

A standard checkpoint is expensive because it blocks all transactions while dirty pages are flushed to disk. A fuzzy checkpoint avoids this by not blocking transactions:

- Write <begin_checkpoint> log record
- Continue executing transactions normally (they may modify pages)
- Flush all dirty pages that were dirty at begin_checkpoint to disk
- Write <end_checkpoint> log record listing dirty pages and their disk addresses
- Recovery uses <end_checkpoint> to determine what needs to be redone — transactions that started after <begin_checkpoint> still need recovery

---

Q: Explain shadow paging as a recovery technique. Describe its structure, commit operation, and recovery procedure. Compare with log-based recovery.

Asked: 2078 Baishakh; 2073 Magh; 2070 Magh | Marks: [4]

Answer

Shadow Paging

Shadow paging is an alternative recovery technique that does not use a log. Instead, it maintains two page tables — a current page table (pointing to the most recent version of each page) and a shadow page table (pointing to a stable, pre-transaction copy of each page). Recovery is achieved by simply reverting to the shadow copy.

Structure

```
                          Before transaction begins:

           Shadow Page Table         Current Page Table        Disk Pages
                ┌─────────────────┐        ┌─────────────────┐
             │ pg1 → disk1  │          │ pg1 → disk1  │ ──────►  [Page 1]
             │ pg2 → disk2  │          │ pg2 → disk2  │ ──────►  [Page 2]
             │ pg3 → disk3  │          │ pg3 → disk3  │ ──────►  [Page 3]
                └─────────────────┘        └─────────────────┘
               (never changed                (changes during transaction)
                                      during transaction)

                     During transaction (modifying page 2):
                           Current page table updated:
         │ pg2 → disk7  │ ──────►  [Page 2 NEW]   (new copy written to disk7)
             Shadow page table still points to disk2 (OLD copy of page 2)
```

Commit Operation

```
To COMMIT a transaction:
  1. Flush all modified pages (current page table entries) to disk
  2. Write the current page table to stable storage
  3. Update the 'db_pointer' (database pointer) to point to the current page
table
     → This single atomic write makes the commit permanent
  4. The old shadow page table can now be discarded (old disk pages freed)

Atomicity of commit: The db_pointer update is a single disk write — atomic on
disk.
Before this write: transaction not committed (shadow is current state)
After this write: transaction committed (current table is the permanent state)
```

Recovery (Abort/Crash)

```
To ABORT a transaction (or recover from crash):
  1. Discard the current page table
  2. Restore the shadow page table as the current page table
  3. Free all disk pages allocated during the transaction (new pages)

Recovery is instantaneous — just reset the db_pointer to the shadow table.
No log scanning needed at all.
```

Shadow Paging vs Log-Based Recovery

Aspect	Shadow Paging	Log-Based Recovery
Mechanism	Two page tables (shadow + current); copy-on-write	Log records written to stable storage; WAL rule
Recovery speed	Instant — just revert to shadow	Must scan and process log records
Commit cost	Must flush ALL modified pages before commit	Only log must be flushed (data can follow later)
Concurrency support	Difficult — hard to share shadow table among concurrent transactions	Excellent — standard for concurrent transactions

Garbage collection	Must free old shadow pages after commit	Log is sequential — easy to truncate
Fragmentation	Data pages become scattered on disk over time	No fragmentation
Used in practice	Rarely (SQLite uses a variant)	Universal standard (PostgreSQL, MySQL, Oracle)

Q: Explain recovery with concurrent transactions using checkpoints. How are the undo-list and redo-list determined?
Asked: 2075 Baisakh; 2071 Magh; 2070 Magh | Marks: [4]

Answer

Recovery with Concurrent Transactions

In practice, many transactions run concurrently and may be active when a crash occurs. The recovery system must handle all of them correctly — redoing committed ones and undoing active ones.

Comprehensive Recovery Algorithm

Step 1: Find the Most Recent Checkpoint

Scan the log backward from the end to find the most recent <checkpoint L> record. L gives the list of transactions that were active at the time of the checkpoint. These transactions were not fully committed at checkpoint time.

Step 2: Build Undo and Redo Lists

```
undo-list = copy of L  (all transactions active at checkpoint)
redo-list = {}

Scan log FORWARD from checkpoint to the end (crash point):
  if <Ti, start> encountered:    add Ti to undo-list
  if <Ti, commit> encountered:   move Ti from undo-list to redo-list
  if <Ti, abort> encountered:    remove Ti from undo-list
```

Step 3: Redo Phase

```
Scan log FORWARD from the checkpoint record:
  For each update record <Ti, Xj, V1, V2> where Ti is in redo-list:
    Set Xj = V2  (re-apply the new value)

This ensures all committed transactions' changes are on disk.
Redo is IDEMPOTENT — applying it multiple times gives the same result.
```

Step 4: Undo Phase

```
Scan log BACKWARD from the crash point (end of log):
```

```
  For each update record <Ti, Xj, V1, V2> where Ti is in undo-list:
    Set Xj = V1  (restore the old value)
    Write a compensation log record <Ti, Xj, V1> to log
                 (CLR — Compensation Log Record)

  When all log records of Ti have been processed:
    Write <Ti, abort> to log

This ensures all uncommitted transactions' changes are rolled back.
```

Worked Example

```
                                  Scenario:
                     Checkpoint taken with active = {T1, T2}
       After checkpoint: T1 commits, T3 starts, T2 and T3 are active at crash

                                Log segment:
                            ... [checkpoint {T1,T2}]
                                <T1, A, 100, 200>
                                   <T3, start>
                                  <T1, commit>
                                <T2, B, 500, 400>
                                <T3, C, 300, 350>
                                  *** CRASH ***

                                Build lists:
                     Start: undo-list = {T1, T2}, redo-list = {}
                        <T3, start>: undo-list = {T1, T2, T3}
                <T1, commit>: redo-list = {T1}, undo-list = {T2, T3}

                    Final: redo-list = {T1}, undo-list = {T2, T3}

                      Redo phase (forward from checkpoint):
                     <T1,A,100,200>: T1 in redo-list → set A=200

                         Undo phase (backward from crash):
                     <T3,C,300,350>: T3 in undo-list → set C=300
                     <T2,B,500,400>: T2 in undo-list → set B=500
                       Write <T2,abort> and <T3,abort> to log
```

SUMMARY TABLE — Chapter 8 Question Coverage

All Chapter 8 questions from TU-IOE BCT examination papers (2070–2082) and their coverage in this document.

No.	Question / Topic	Asked In	Marks
C1	Failure classification — transaction, system crash, disk failure (with examples)	ALL years 2070–2082	[4]
C2	Storage structure — volatile, non-volatile, stable storage	2079 Ashwin, 2079 Jestha, 2078 Poush, 2076, 2075 Bhadra, 2075 Baisakh, 2073 Magh, 2071 Magh, 2070 Magh	[4]
C3	Log-based recovery — log records, write-ahead logging (WAL) rule	All years 2070–2082	[4]
C4	Deferred database modification — redo only, example trace	2081 Chaitra, 2079 Chaitra, 2079 Jestha, 2078 Chaitra, 2078 Baishakh, 2076, 2074, 2072 Ashwin, 2071, 2070	[4]
C5	Immediate database modification — undo and redo, example trace	2082, 2081 Chaitra, 2080, 2079 Ashwin, 2078 Chaitra, 2077, 2075 Baisakh, 2075 Bhadra, 2073 Magh, 2072 Magh, 2070 Magh	[4]
C6	Checkpoints — why needed, fuzzy checkpoint, recovery with checkpoints	2081 Ashwin, 2080, 2079 Ashwin, 2078 Poush, 2077, 2075 Baisakh, 2074, 2071 Magh	[4]
C7	Shadow paging — page table, shadow page table, recovery	2078 Baishakh, 2073 Magh, 2070 Magh	[4]
C8	Recovery with concurrent transactions — undo-list, redo-list, checkpoints	2075 Baisakh, 2071 Magh, 2070 Magh	[4]

Key notation: <Ti,start> = start record; <Ti,Xj,V1,V2> = Ti updates Xj from V1 to V2; <Ti,commit> = commit record; <checkpoint L> = checkpoint with active list L. WAL rule: log record before disk write (for undo); all logs before commit (for redo). Deferred: redo only. Immediate: undo (active) + redo (committed). Checkpoint scan: undo-list starts as L; redo-list starts empty; updated by scanning forward to crash.

— End of Chapter 8 Answer Bank —